Universidad Nacional Autónoma de México

Facultad de Ingeniería

Bases de Datos

Tarea:
Tipos de datos en PostgreSQL

Numéricos, carácter, fecha y otros tres tipos:
ejemplos, capacidad y formato de representación

Alumno: García Olvera José Ignacio

Fecha: 2 de noviembre de 2026

1. Introducción

Al definir cada columna de una tabla es indispensable indicar su tipo de dato, el cual debe reflejar la naturaleza de los datos (montos, fechas, cantidades, nombres). El estándar SQL define los tipos que todo manejador debería implementar, aunque ninguno lo cumple al 100 %. Se clasifican en cadenas (de caracteres y binarias), numéricos (exactos y aproximados), fechas y otros (`boolean`, `XML`, etc.). A continuación se describen los de PostgreSQL.

2. Tipos numéricos

2.1. Exactos

Representan enteros o números con parte decimal finita. Para los decimales se definen la **precisión** p (total de dígitos) y la **escala** s (dígitos decimales), de modo que la parte entera tiene $p - s$ dígitos. Por ejemplo, `numeric(10,4)` admite 6 dígitos enteros y 4 decimales: $[-999999.9999,\ 999999.9999]$.

Tabla 1: Tipos numéricos exactos en PostgreSQL.

Tipo	Tamaño	Rango / capacidad
`smallint`	2 B	$-32\,768$ a $32\,767$
`integer`	4 B	$-2\,147\,483\,648$ a $2\,147\,483\,647$
`bigint`	8 B	$\pm 9.22 \times 10^{18}$
`numeric(p,s)` / `decimal(p,s)`	variable	Hasta $131\,072$ dígitos enteros y $16\,383$ decimales
`serial` / `bigserial`	4 B / 8 B	Enteros autoincrementales desde 1

Formato: los enteros se guardan en binario de tamaño fijo y se muestran como enteros decimales (`2026`). `numeric` es de longitud variable y se muestra con exactamente s decimales; si sobran decimales se redondea y si la parte entera excede $p - s$ dígitos hay error.

2.2. Aproximados

Se usan cuando no se conoce con exactitud la precisión de los valores. PostgreSQL ofrece `real` (4 B, ≈6 dígitos) y `double precision` (8 B, ≈15 dígitos), basados en el estándar IEEE 754, por lo que algunos valores sólo se representan de forma aproximada.

```sql
create table producto (
  producto_id serial primary key,
  existencias smallint,
  precio numeric(10,4),
  peso double precision
);
insert into producto (existencias, precio, peso) values (25, 1234.56789, 0.1);
-- precio se guarda como 1234.5679 (redondeado a 4 decimales)
select 0.1::double precision + 0.2;   -- 0.30000000000000004 (aproximado)
```

3. Tipos de carácter

Una cadena es una secuencia de caracteres de un juego de caracteres (ASCII, ISO 8859, UTF-8, etc.); el número de bytes que ocupa depende de esa codificación. Por ejemplo, la letra `a` es `0x61` en ISO-8859-1 y en UTF-8, pero `0x00 0x61` en UTF-16.

- `char(n)`: longitud fija; si la cadena es menor que n se rellena con espacios.
- `varchar(n)`: longitud variable; sólo guarda los caracteres reales hasta un máximo de n.
- `text`: longitud variable sin límite declarado (equivale al `clob` del estándar).

Capacidad: n puede llegar a 10 485 760 caracteres y la cadena más larga posible es de aproximadamente 1 GB. **Formato:** los valores se escriben entre comillas simples y la longitud se mide en caracteres, no en bytes.

```
create table cliente (
  rfc char(13),
  nombre varchar(40),
  biografia text
);
insert into cliente values ('GAOJ000101AB1', 'José Ignacio', 'Estudiante de la FI');
select char_length(nombre), octet_length(nombre) from cliente;
-- 12 caracteres, 13 bytes ('é' ocupa 2 bytes en UTF-8)
```

4. Tipos de fecha y hora

El estándar define `date` (año, mes, día), `time` (HH:MM:SS) y `timestamp` (fecha y hora). PostgreSQL los implementa todos, además de `interval`, a diferencia de Oracle, que usa `date` para todo.

Tabla 2: Tipos de fecha y hora en PostgreSQL.

Tipo	Tamaño	Rango	Resolución
`date`	4 B	4713 a. C. a 5 874 897 d. C.	1 día
`time`	8 B	00:00:00 a 24:00:00	1 μs
`timestamp`	8 B	4713 a. C. a 294 276 d. C.	1 μs
`interval`	16 B	$\pm$178 000 000 años	1 μs

Formato: internamente las fechas se guardan como números, no como cadenas. Por defecto PostgreSQL las muestra en formato ISO 8601 (`2026-11-02`, `2026-11-02 14:30:15`) y con `to_char` se puede usar otro formato.

```
select date '2026-08-10' + interval '4 months' as fin,
       to_char(date '2026-08-10', 'DD/MM/YYYY') as formateada;
-- 2026-12-10 00:00:00 | 10/08/2026
```

5. Otros tres tipos de datos

`boolean`. Agregado en SQL:2003, con valores `TRUE` y `FALSE`. Oracle no lo tiene (usa `NUMBER(1,0)`), pero PostgreSQL sí. Ocupa 1 byte, acepta `true`/`false`, `yes`/`no`, `1`/`0` y se muestra como `t` o `f`.

`bytea`. Equivale al `blob` del estándar y guarda datos binarios (imágenes, documentos, audio). Ocupa el tamaño de los datos más 1 o 4 bytes, hasta 1 GB por valor. Se muestra en hexadecimal con el prefijo `\x`, por ejemplo `\x89504e47`.

xml. Guarda documentos XML y verifica que estén bien formados, lo que permite consultarlos desde SQL con XPath. Es de longitud variable (hasta 1 GB) y se crea con `XMLPARSE` o con `'...'::xml`.

```
create table factura (
  pagada  boolean default false,
  archivo bytea,
  datos   xml
);
insert into factura values (true, '\x89504E47', '<factura><total>1160.00</total></factura>')
    ;
select pagada, archivo, xpath('/factura/total/text()', datos) from factura;
-- t | \x89504e47 | {1160.00}
```

Referencias

[1] Rodríguez C., J. A. *Tema 04. Modelo relacional*, sección 4.5 "Tipos de datos empleados para realizar modelos relacionales". Material de apoyo, Facultad de Ingeniería, UNAM.

[2] The PostgreSQL Global Development Group. *PostgreSQL Documentation, Chapter 8: Data Types*. `https://www.postgresql.org/docs/current/datatype.html`